```sql
-- Find the total number of products sold by each store along with the store name.

Select stores.store_name,
SUM(order_items.quantity) products_sold
From stores Left Join orders using (store_id)
Left Join order_items using (order_id)
Group by stores.store_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

store_name	products_sold
Santa Cruz Bikes	1516
Baldwin Bikes	4779
Rowlett Bikes	783

```sql
-- Calculate the cumulative sum of quantities sold for each product over time.

with cte as (select products.product_id, products.product_name,
orders.order_date,
sum(order_items.quantity) total_quantity
from products join order_items
using (product_id)
join orders using (order_id)
group by products.product_id, products.product_name,
orders.order_date)

select* , sum(total_quantity) over (partition by product_id order by order_date) cum_quantity
from cte;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

product_id	product_name	order_date	total_quantity	cum_quantity
2	Ritchey Timberwolf Frameset - 2016	2016-01-03	2	2
2	Ritchey Timberwolf Frameset - 2016	2016-01-14	2	4
2	Ritchey Timberwolf Frameset - 2016	2016-01-18	1	5
2	Ritchey Timberwolf Frameset - 2016	2016-02-05	1	6
2	Ritchey Timberwolf Frameset - 2016	2016-02-09	1	7
2	Ritchey Timberwolf Frameset - 2016	2016-02-26	1	8
2	Ritchey Timberwolf Frameset - 2016	2016-02-28	2	10

```sql
-- Find the product with the highest total sales (quantity * price) for each category.

with a as (select categories.category_id, categories.category_name, products.product_id,
products.product_name,
sum(order_items.quantity*order_items.list_price) sales
from products join order_items using(product_id)
join categories using(category_id)
group by categories.category_id, categories.category_name, products.product_id,
products.product_name)

select* from
(select *, dense_rank() over(partition by category_id order by sales desc)rnk from a) b
where rnk= 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category_id	category_name	product_id	product_name	sales	rnk
1	Children Bicycles	23	Electra Girl's Hawaii 1 (20-inch) - 2015/2016	4619846.00	1
2	Comfort Bicycles	26	Electra Townie Original 7D EQ - 2016	8039866.00	1
3	Cruisers Bicycles	16	Electra Townie Original 7D EQ - 2016	9359844.00	1
4	Cyclocross Bicycles	11	Surly Straggler 650b - 2016	25382949.00	1
5	Electric Bikes	9	Trek Conduit+ - 2016	43499855.00	1
6	Mountain Bikes	7	Trek Slash 8 275 - 2016	61599846.00	1
7	Road Bikes	56	Trek Domane SLR 6 Disc - 2017	23649957.00	1

```sql
-- Find the customer who spent the most money on orders.

with a as (
select customers.customer_id, customers.first_name, customers.last_name,
sum(order_items.quantity*order_items.list_price) total_spent
from customers join orders using(customer_id)
join order_items using(order_id)
group by customers.customer_id, customers.first_name, customers.last_name)

select * from
(select *, dense_rank() over(order by total_spent desc) rnk from a) b
where rnk = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	first_name	last_name	total_spent	rnk
10	Pamelia	Newman	3780184.00	1

```sql
-- Find the highest-priced product for each category name.

with a as (
select categories.category_id, categories.category_name,
products.product_id, products.product_name, products.list_price
from products join categories using(category_id))

select * from
(select *, dense_rank() over(partition by category_id order by list_price desc) rnk from a) b
where rnk = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category_id	category_name	product_id	product_name	list_price	rnk
1	Children Bicycles	98	Electra Straight 8 3i (20-inch) - Boy's - 2017	48999.00	1
1	Children Bicycles	100	Electra Townie 3i EQ (20-inch) - Boys' - 2017	48999.00	1
1	Children Bicycles	280	Trek Superfly 24 - 2017/2018	48999.00	1
2	Comfort Bicycles	303	Electra Townie Go! 8i - 2017/2018	259999.00	1
3	Cruisers Bicycles	251	Electra Townie Commute Go! - 2018	299999.00	1
3	Cruisers Bicycles	252	Electra Townie Commute Go! Ladies' - 2018	299999.00	1
4	Cyclocross Bicycles	207	Trek Boone 7 Disc - 2018	399999.00	1
5	Electric Bikes	61	Trek Powerfly 8 FS Plus - 2017	499999.00	1
5	Electric Bikes	203	Trek Powerfly 7 FS - 2018	499999.00	1
5	Electric Bikes	205	Trek Super Commuter + 8S - 2018	499999.00	1
6	Mountain Bikes	43	Trek Fuel EX 98 275 Plus - 2017	529999.00	1
6	Mountain Bikes	47	Trek Remedy 98 - 2017	529999.00	1
7	Road Bikes	155	Trek Domane SLR 9 Disc - 2018	1199999.00	1

```
-- Find the total number of orders placed by each customer per store.

select customers.first_name, customers.last_name,
stores.store_name, count(orders.order_id) total_orders
from customers join orders using(customer_id)
join stores using(store_id)
group by customers.first_name, customers.last_name, stores.store_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

first_name	last_name	store_name	total_orders
Johnathan	Velazquez	Santa Cruz Bikes	1
Nova	Hess	Santa Cruz Bikes	2
Neil	Mccall	Santa Cruz Bikes	2
Marvin	Mullins	Santa Cruz Bikes	2
Maribel	William	Santa Cruz Bikes	1
Lea	Key	Santa Cruz Bikes	1
Sindy	Anderson	Santa Cruz Bikes	1
Lanita	Burton	Santa Cruz Bikes	1
Norine	Huffman	Santa Cruz Bikes	1
Randee	Pitts	Santa Cruz Bikes	1
Tangela	Hurley	Santa Cruz Bikes	1
Drucilla	Gilliam	Santa Cruz Bikes	1
Kimbery	Nieves	Santa Cruz Bikes	1
Darren	Witt	Santa Cruz Bikes	1
Twana	Arnold	Santa Cruz Bikes	1
Joesph	Delacruz	Santa Cruz Bikes	2
Denis	Logan	Santa Cruz Bikes	2
Cesar	Wilkins	Santa Cruz Bikes	1

```
-- Find the names of staff members who have not made any sales.

select staffs.*
from staffs
left join orders using(staff_id)
where order_id is null;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

staff_id	first_name	last_name	email	phone	active	store_id	manager_id
1	Fabiola	Jackson	fabiolajackson@bikesshop	(831) 555-5554	1	1	NULL
4	Virgie	Wiggins	virgiewiggins@bikesshop	(831) 555-5557	1	1	2
5	Jannette	David	jannettedavid@bikesshop	(516) 379-4444	1	2	1
10	Bernardine	Houston	bernardinehouston@bikesshop	(972) 530-5557	1	3	7

```sql
-- Find the top 3 most sold products in terms of quantity.

with a as (
select products.product_id, products.product_name,
sum(order_items.quantity) total_quantity
from products join order_items using(product_id)
group by products.product_id, products.product_name)

select * from
(select *, dense_rank() over(order by total_quantity desc) rnk from a) b
where rnk <= 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	total_quantity	rnk
6	Surly Ice Cream Truck Frameset - 2016	167	1
13	Electra Cruiser 1 (24-Inch) - 2016	157	2
16	Electra Townie Original 7D EQ - 2016	156	3

```sql
-- Find the median value of the price list.

with a as (
select list_price,
row_number() over(order by list_price) rn,
count(*) over() total_count
from products)

select list_price
from a
where rn in ((total_count+1)/2, (total_count+2)/2);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

list_price
74999.00

```sql
-- List all products that have never been ordered.(use Exists)

with a as (
    select p.product_id, p.product_name
    from products p
    where not exists (
        select 1
        from order_items oi
        where oi.product_id = p.product_id
    )
)
select *
from a;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name
1	Trek 820 - 2016
121	Surly Krampus Frameset - 2018
125	Trek Kids' Dual Sport - 2018
154	Trek Domane SLR 6 Disc Women's - 2018
195	Electra Townie Go! 8i Ladies' - 2018
267	Trek Precaliber 12 Girl's - 2018
284	Electra Savannah 1 (20-inch) - Girl's - 2018
291	Electra Sweet Ride 1 (20-inch) - Girl's - 2018
316	Trek Checkpoint ALR 4 Women's - 2019
317	Trek Checkpoint ALR 5 - 2019
318	Trek Checkpoint ALR 5 Women's - 2019

```sql
-- List the names of staff members who have made more sales than the average number of sales by all staff members.

with a as (
    select
        s.staff_id,
        s.first_name,
        s.last_name,
        count(o.order_id) total_sales,
        avg(count(o.order_id)) over() avg_sales
    from staffs s
    left join orders o using(staff_id)
    group by s.staff_id, s.first_name, s.last_name
)
select first_name, last_name, total_sales
from a
where total_sales > avg_sales;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	last_name	total_sales
Mireya	Copeland	164
Genna	Serrano	184
Marcelene	Boyer	553
Venita	Daniel	540

Limit to 1000 rows

```sql
-- Identify the customers who have ordered all types of products (i.e., from every category).

with a as (
    select customers.customer_id,
    count(distinct products.category_id) categories_ordered
    from customers
    join orders using(customer_id)
    join order_items using(order_id)
    join products using(product_id)
    group by customers.customer_id
),
b as (
    select count(*) total_categories
    from categories
)
select customers.customer_id, customers.first_name, customers.last_name
from customers join a using(customer_id)
cross join b
where a.categories_ordered = b.total_categories;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	first_name	last_name
9	Genoveva	Baldwin